User Ratings of Video Games on Amazon

Grace Boye-Williams, Will Pierce, Corey Rogers, Jude Shin

Data Description and Importance

Data Description: Amazon reviews dataset collected in 2023 by McAuley lab

- Specifically Video Game reviews
  - 2.8M Users, 137.2k Items, and 4.6M Ratings
- User Reviews:
  - **rating**: float rating of product from 1.0 to 5.0
  - **parent_asin**: parent ID of product being reviewed
  - **user_id**: unique identifier of reviewer

Importance:

- Better recommendations can increase user engagement and sales
- Personalized experiences improve customer satisfaction and retention

Initial Parsing

Parsing (both reviews and metadata dataset):

1) Json text file input loaded from disk to RDD
2) .map() converting to a tuple
3) .filter() removing bad parsed tuples
4) .map() to key-value format (parent_asin, (relevant data tuple))

Note:

- Bad json parsing in step (3) results in a ‘null’ instead of a tuple
- Key-value pairs needed for the .join() on both datasets

Joining the Datasets

- RDD at this point comes in as **(parent_asin, (reviews tuple))** from the parsed reviews dataset, and **(parent_asin, (metadata tuple))**.
- .join() produces a compound tuple of the form:
  (parent_asin, ((reviews tuple), (metadata tuple))).
- One final .map() to format the tuples for the algorithms:
  ((user_id, parent_asin), rating)

- Tuples saved to disk in order to skip the computation of the RDDs every time.

Formulas Used

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

$$\text{Pearson Correlation}(A, B) = \frac{(A - \bar{A}) \cdot (B - \bar{B})}{\|A - \bar{A}\| \|B - \bar{B}\|}$$

Cosine: Compares rating vectors directly

Pearson: Compares rating patterns relative to each user’s average rating

Algorithms Implemented

Pearson's Correlation

Input: 2 user's rating data

Output: Correlation value in the range [-1, 1]

Implementation:

1. Find all co-rated items (if none, return 0)
2. Find each user's mean rating of co-rated items
3. Find difference between each user's item rating and mean rating
4. Compute dot product of the differences
5. Divide the dot product by magnitude of the differences (return 0 if magnitude 0)

Cosine Similarity

Input: Two non-zero vectors (user ratings in this instance)

Output: Similarity score in the range [-1, 1]

Implementation:

1. Compute dot product
2. Compute magnitude of each vector
3. Divide by product of magnitudes
4. Return zero if magnitude is zero

Execution Layer

Rating Prediction

Input: Full data, target user and product, similarity function (Pearson / cosine), top-n neighbors parameter

Output: Predicted rating as a double

Implementation:

1. Builds user profiles: user -> list of (product, rating)
2. Computes similarity between target user and all other users
3. Selects users who rated the target product
4. Filters to top N most similar positive neighbors
5. Predicts rating using weighted average of neighbor ratings

Job Driver

Output: Prints and writes results (expected, actual) to file

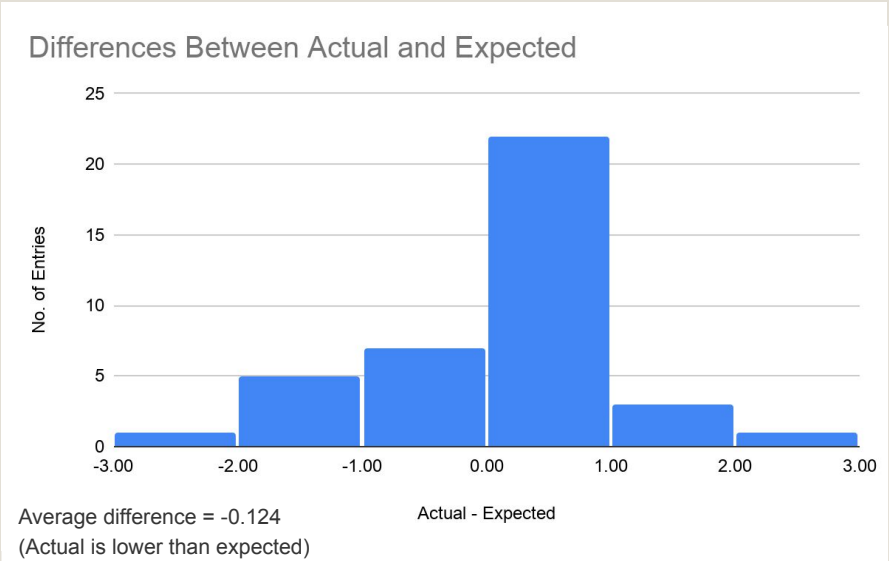
Implementation:

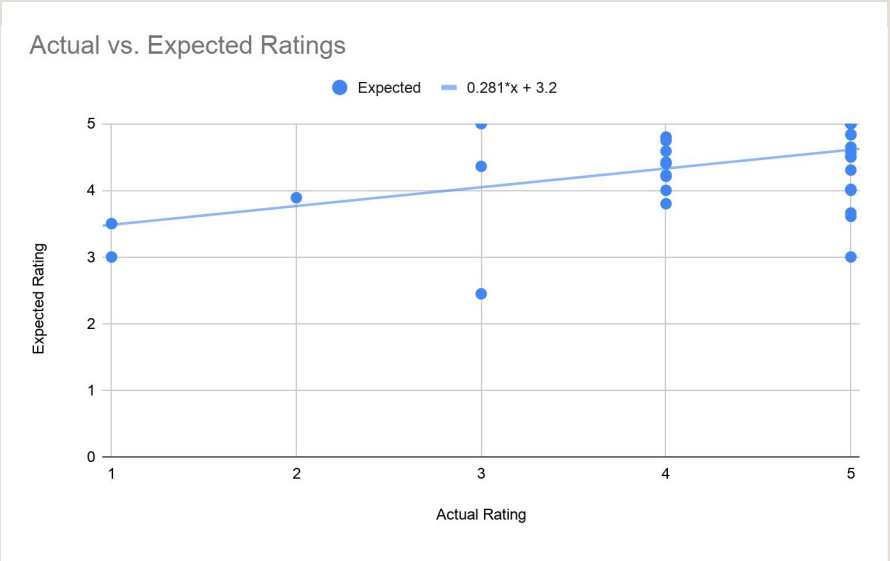
1. Reads processed dataset
2. Configures top-n and selects lines to read
3. Loop:
    a. Extracts data line from dataset
    b. Calls prediction program for that user and item
    c. Prints and writes result

User Rating Predictions and Results

- If a user and object review did not find any comparable neighbors, we gave the expected = 0.0 and excluded them from the tables
- 39 out of 1000 lines returned non-zero expected values

Expected Range	Count
2.0–2.99	1
3.0–3.99	7
4.0–4.49	10
4.5–4.99	13
5	8





Conclusions

1. Correlation

Given our test runs of the data set, there is a clear correlation between the rating our algorithm computed and the actual rating given by the user.

- Scatterplot showed correlation, trendline was positive
- 22/39 predictions were very close to the actual review

2. Accuracy

Despite the correlation, the algorithm often expected users to give slightly higher reviews than they would actually give.

We could attribute that to:

- Our implementation
- The nature of Amazon reviews
- Small sample size and randomness

3. Improvement

- 4% of reviews were able to be compared
- Currently only makes direct connections between 2 users
- Being able to make inferences using groups instead of individuals would potentially improve accuracy and utilization of data

Challenges Faced

- Determining how to join data
  - Both cartesian product was used, as well as the .join() function.
  - Join **case** statement was significantly slower if the case statement tuples were very large
  - ex) ((_, _, _, _, _, _, rating, parent_asin, _), (_, _, _, _)) could be simplified to (tuple_1, tuple_2).

- Figuring out interesting/useful metrics
  - The .join() allows us to include other information like the price of the product that was being reviewed, or the average rating at the time the user reviewed a product.
  - These algorithms could be extrapolated to handle other pieces of information in the dataset like the pieces mentioned above.